Metodologie di Programmazione (M-Z)

II semestre - a.a. 2022 – 2023

Parte 5 – Ereditarietà**

a cura di Mattia Samory*



SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

📣Avvisi

- Niente lezione il **19/4**
- Niente lezione neanche il 28/3 e 2/4
- Prove in itinere (?)

📚 Letture

- ereditarietà (Horstmann 9.1–9.3)
- The missing semester of your CS education

🔄Recap

- enum
- stack, heap, metaspace

enum

- Insieme finito di valori
- Esempio: enum Size { SMALL, MEDIUM, LARGE }
- Uso tipico:

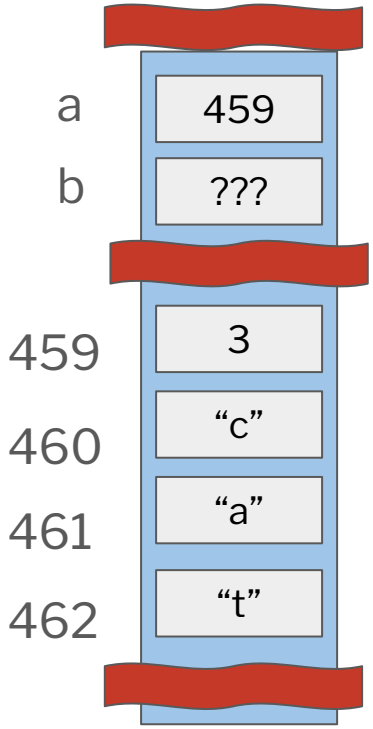
```
Size imageSize = Size.MEDIUM;
…
if (imageSize == Size.SMALL)
```

- Approccio più robusto rispetto a costanti intere

```
public static final int SMALL = 1;
public static final int MEDIUM = 2;
public static final int LARGE = 3;
```

👐TPS

- quali porzioni della memoria sono rappresentata nel grafico a destra (stack, heap, o metaspace)?
- cosa contiene la locazione di memoria corrispondente a b al termine delle seguenti due chiamate?



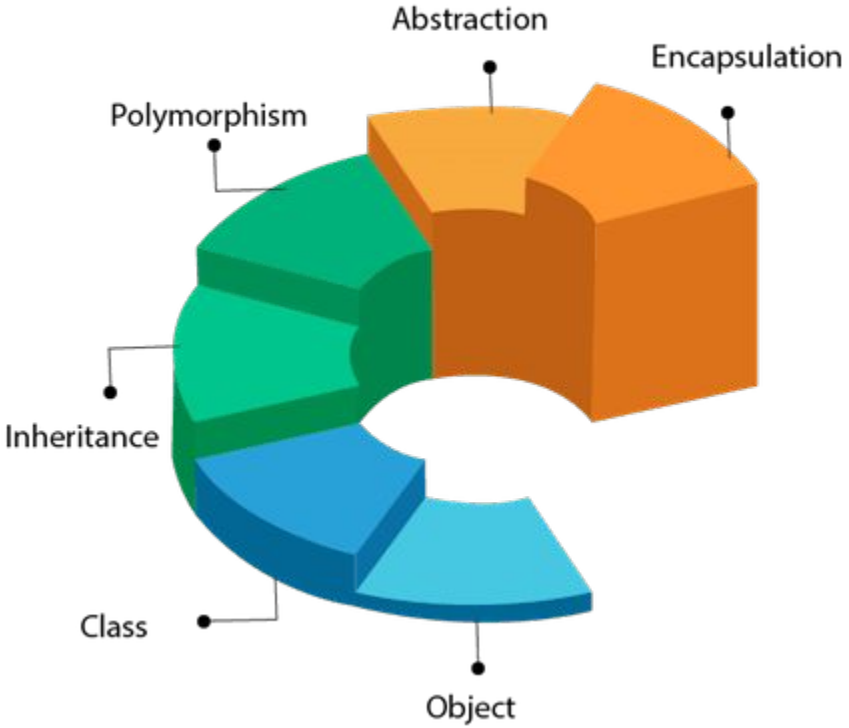

```
String a = new String(“cat”);
String b = a;
```

🎯Obiettivi

- ereditarietà
- classi/metodi astratti
- super (vs. this)
- overload/override
- is-a vs. has-a

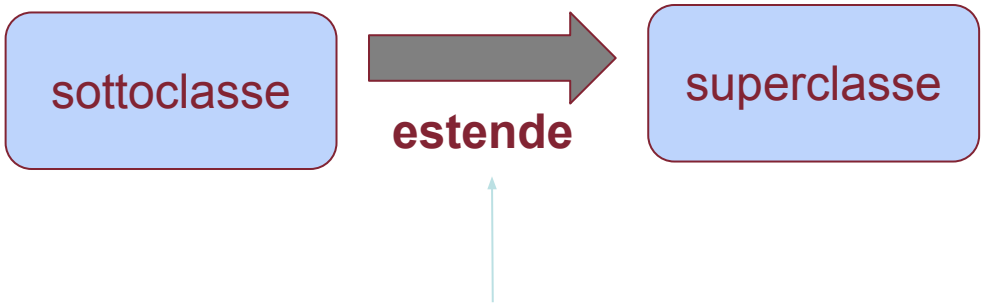
Ereditarietà

Ereditarietà: creazione di relazioni di gerarchia tra le classi



sottoclasse

Java supporta un meccanismo di ereditarietà noto come subclassing**. Sottoclassi** permettono al programmatore di cambiare funzionamento ed aggiungere funzionalità senza riscrivere la classe da zero.

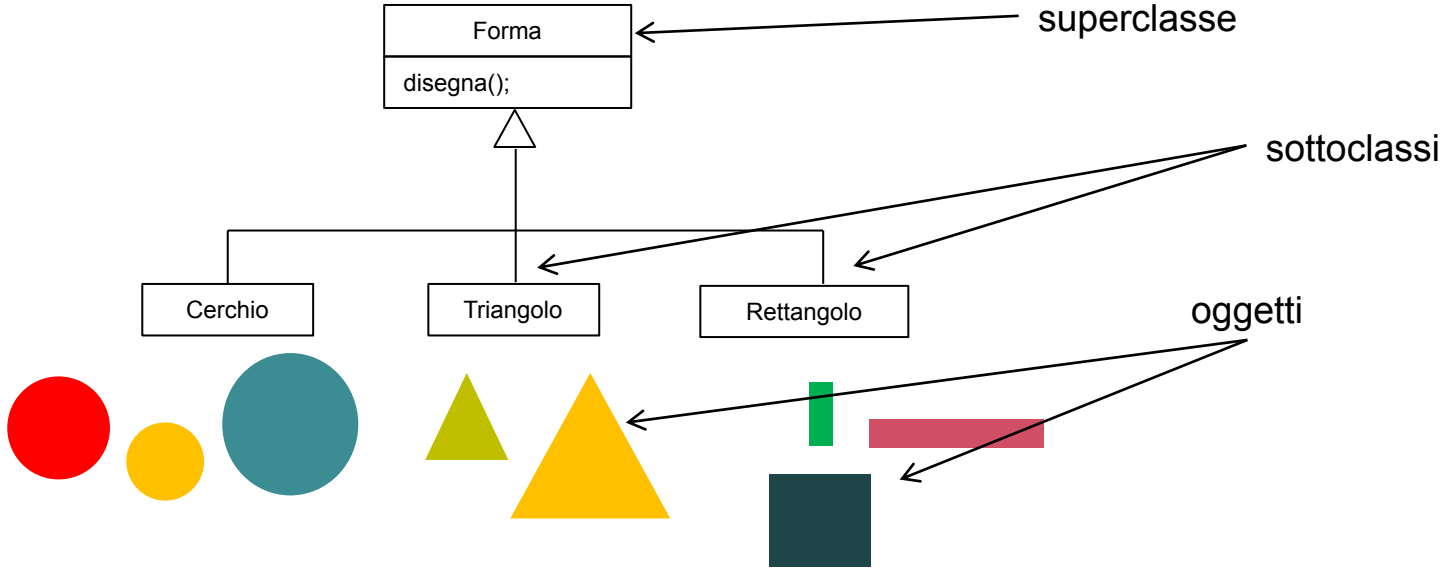


Ereditarietà

- Un concetto cardine della programmazione orientata agli oggetti
- Una forma di riuso del software in cui una classe è creata:
  - "assorbendo" i membri di una classe esistente
  - aggiungendo nuove caratteristiche o migliorando quelle esistenti
- Programmazione mattone su mattone
  - Detto anche: non si butta via niente
- Aumenta le probabilità che il sistema sia implementato e manutenuto in maniera efficiente

Molti tipi di “forma”

- Si può progettare una classe Forma che rappresenta una forma generica e poi specializzarla estendendo la classe



Molti tipi di “forma”

Estende la classe Forma

```java
public class Forma
{
    public void disegna() { }
}
```

```java
public class Triangolo extends Forma
{
    private double base;
    private double altezza;

    public Triangolo(double base, double altezza)
    {
        this.base = base;
        this.altezza = altezza;
    }

    public double getBase()
    {
        return base;
    }

    public double getAltezza()
    {
        return altezza;
    }
}
```

```java
public class Cerchio extends Forma
{
    /**
     * Raggio del cerchio
     */
    private double raggio;

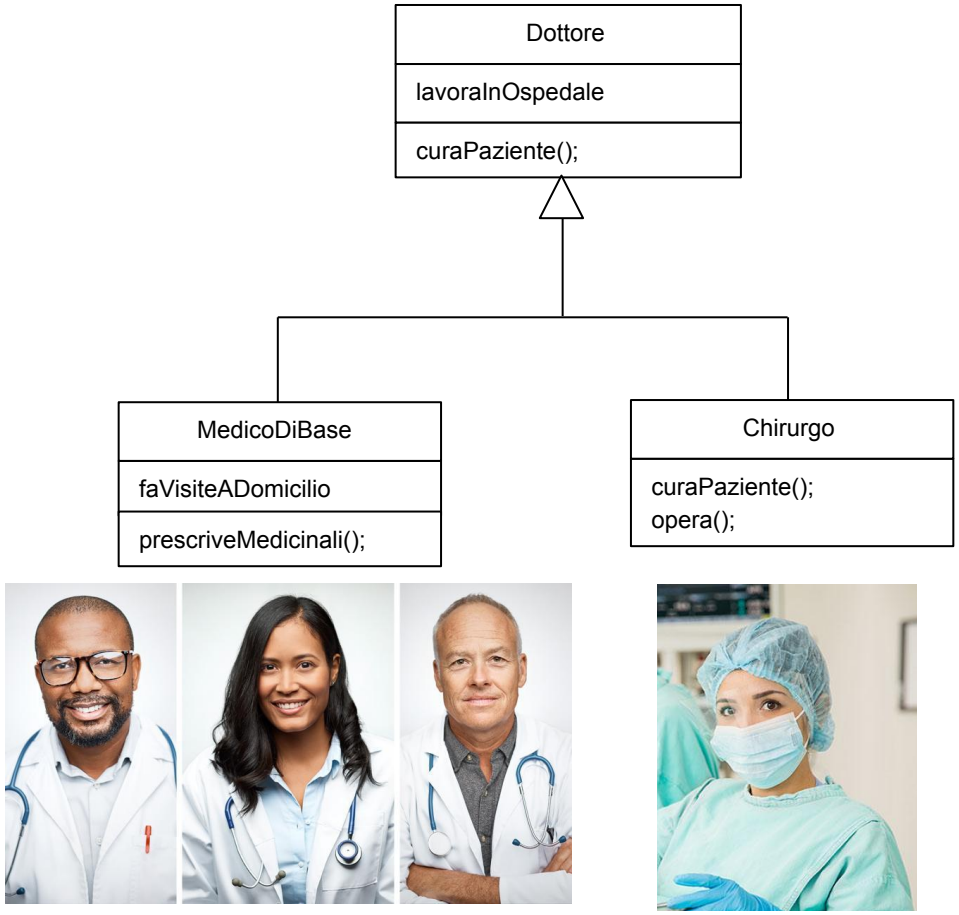
    public Cerchio(int raggio)
    {
        this.raggio = raggio;
    }

    public double getRaggio() { return raggio; }
    public double getCirconferenza() { return 2*Math.PI*raggio; }
}
```

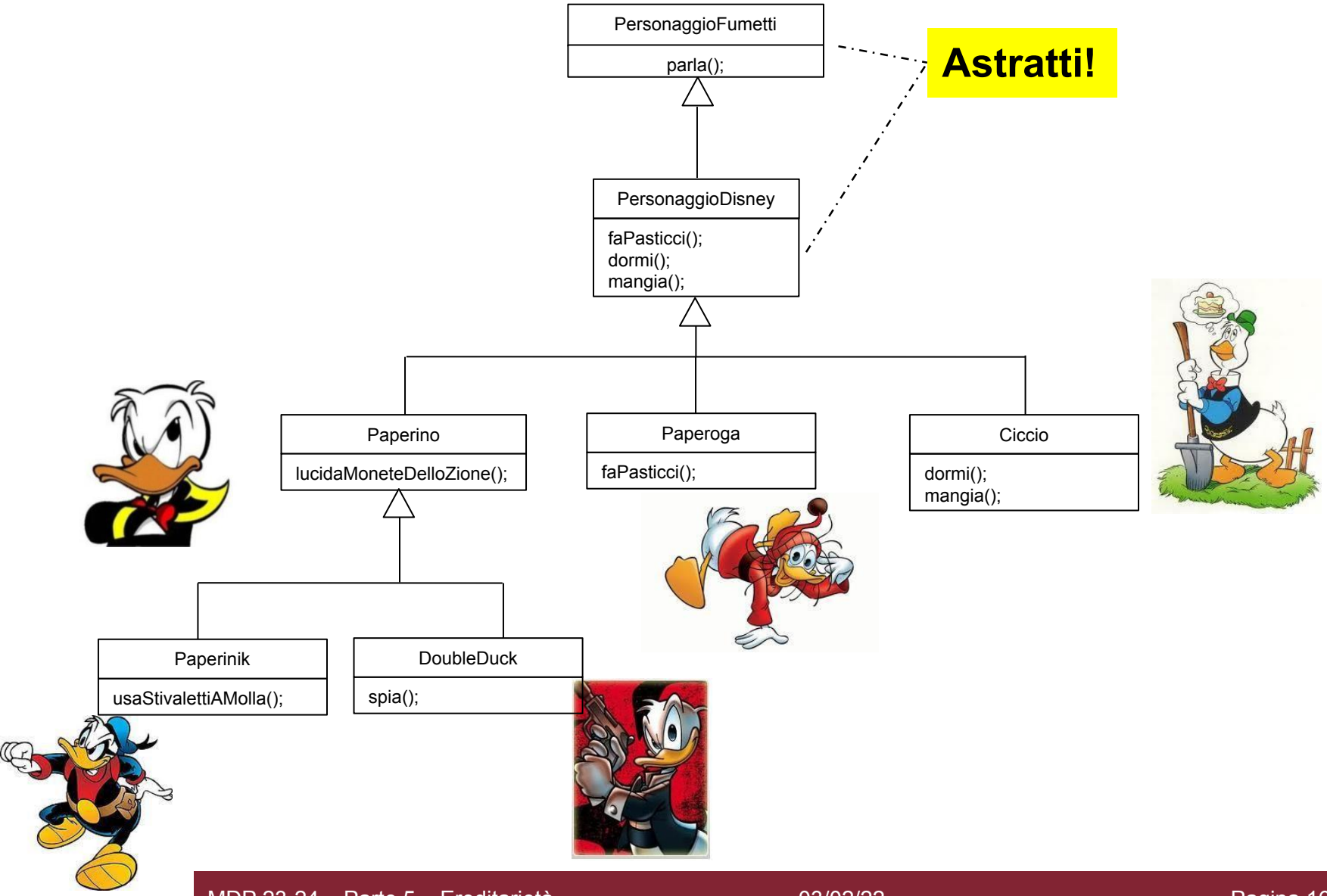
Ereditarietà: che cosa si eredita?

- Una sottoclasse estende la superclasse
- La sottoclasse eredita i membri della superclasse
  - campi e metodi d'istanza secondo il livello di accesso specificato
- Inoltre la sottoclasse può:
  - aggiungere nuovi metodi e campi
  - ridefinire i metodi che eredita dalla superclasse (tipicamente **NON** i campi)

Esempio: il dottore, il medico di base e il chirurgo



Esempio: paperino & company



Esempio: Hokuto No Ken







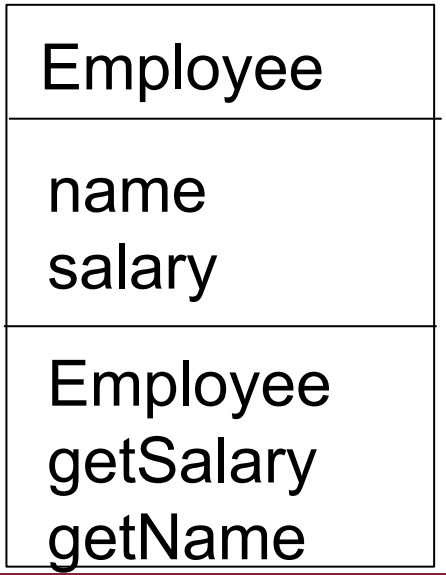


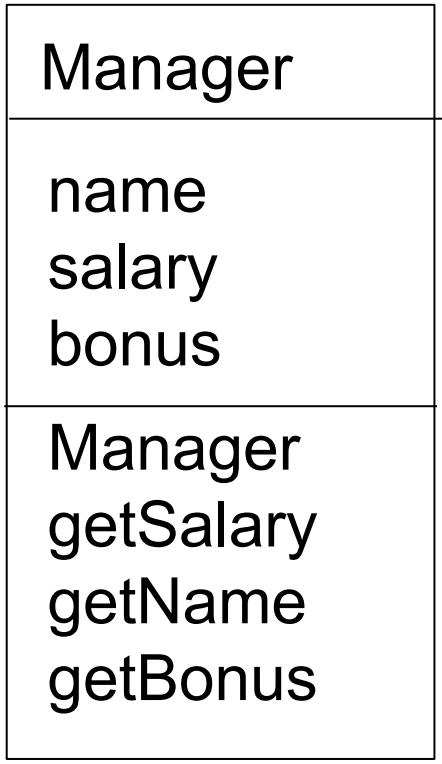


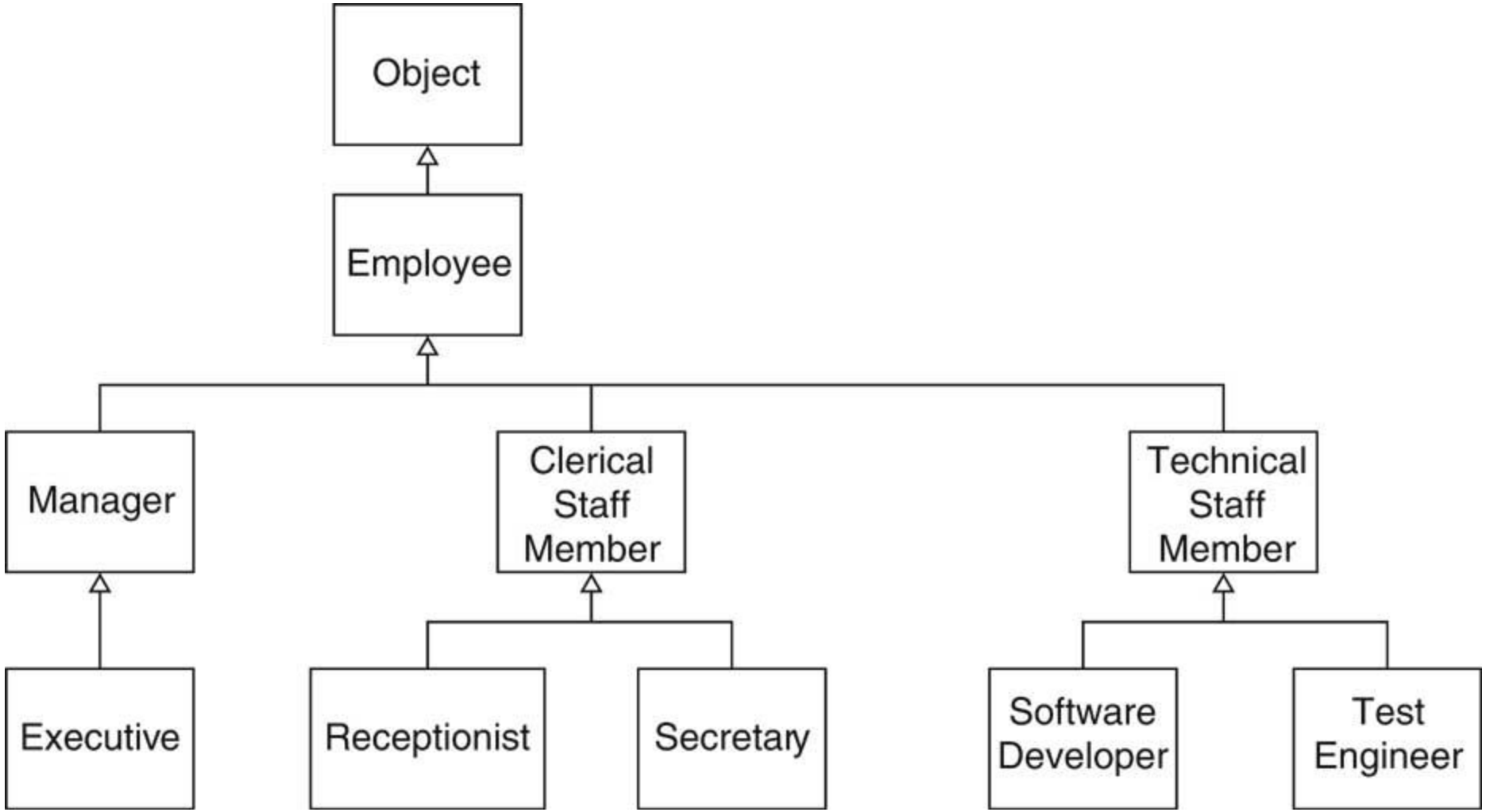
👐TPS

Vi è stato chiesto di sviluppare un'applicazione per gestire le risorse umane di un'azienda. Considerate le classi **`Employee`** e **`Manager`**.

- quale delle due è la superclasse?






Object
Employee
Manager
Clerical Staff Member
Technical Staff Member
Executive
Receptionist
Secretary
Software Developer
Test Engineer

Il principio di sostituzione (Liskov – L in SOLID)

Dovrebbe sempre essere possibile usare una sottoclasse al posto della superclasse

- Per esempio:

```
Employee e;
…
System.out.println("salary=" + e.getSalary());
```

- e potrebbe essere un riferimento di tipo Manager
- Polimorfismo: viene chiamata l'implementazione corretta di getSalary

🎯Obiettivi

- ✅ ereditarietà
- ☐ classi/metodi astratti
- ☐ super (vs. this)
- ☐ overload/override
- ☐ is-a vs. has-a

Classi astratte

- Una classe astratta (definita mediante la parola chiave abstract) non può essere istanziata
  - Quindi **NON** possono esistere oggetti per quella classe

```
/**
 * Classe astratta: non e' possibile istanziarla
 */
public abstract class PersonaggioDisney
{
    /**
     * Metodo astratto senza implementazione
     */
    abstract void faPasticci();
}
```

Classi astratte

- Una classe astratta (definita mediante la parola chiave abstract) non può essere istanziata
  - Quindi **NON** possono esistere oggetti per quella classe

```
/**
 * Classe astratta: non e' possibile istanziarla
 */
public abstract class PersonaggioDisney
{
    /**
     * Metodo astratto senza implementazione
     */
    abstract void faPasticci();
}
```

- Tipicamente verrà estesa da altre classi, che invece potranno essere istanziate

```
public class Paperoga extends PersonaggioDisney
{
    public void faPasticci()
    {
        System.out.println("bla bla bla bla bla bla bla");
    }
}
```

Metodi astratti

- Anche i metodi possono essere definiti astratti
  - Esclusivamente all'interno di una classe dichiarata astratta
- **NON** forniscono l'implementazione per quel metodo

```
/**
 * Classe astratta: non e' possibile istanziarla
 */
public abstract class PersonaggioDisney
{
    /**
     * Metodo astratto senza implementazione
     */
    abstract void faPasticci();
}
```

Metodi astratti

- Anche i metodi possono essere definiti astratti
  - Esclusivamente all'interno di una classe dichiarata astratta
- **NON** forniscono l'implementazione per quel metodo

```
/**
 * Classe astratta: non e' possibile istanziarla
 */
public abstract class PersonaggioDisney
{
    /**
     * Metodo astratto senza implementazione
     */
    abstract void faPasticci();
}
```

- Impongono alle sottoclassi non astratte di implementare il metodo

```
public class Paperoga extends PersonaggioDisney
{
    public void faPasticci()
    {
        System.out.println("bla bla bla bla bla bla bla");
    }
}
```

🎯Obiettivi

- ✅ ereditarietà
- ✅ classi/metodi astratti
- ☐ super (vs. this)
- ☐ overload/override
- ☐ is-a vs. has-a

chiamare metodi della superclasse

- Non possiamo accedere a campi privati della superclasse

```
public class Manager extends Employee{
   public double getSalary()   {
      return salary + bonus;  // ERROR--private field
   }
   …
```

- Attenzione quando si chiama un metodo della superclasse

```
public double getSalary(){
   return getSalary() + bonus;  // ERROR--recursive call
}
```

super

```
public double getSalary(){
   return super.getSalary() + bonus;
}
```

- super non è un riferimento
- disabilita i meccanismi di polimorfismo

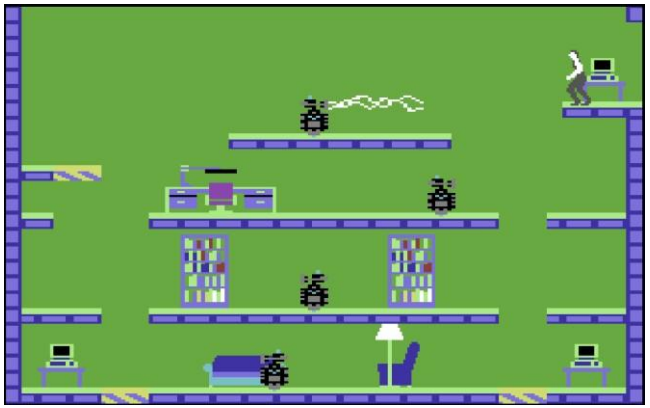
super nei costruttori

```
public Manager(String aName)
 {
    super(aName); // calls superclass constructor
    bonus = 0;
 }
```

- Dev'essere la prima espressione nel costruttore della sottoclasse
- Se la sottoclasse non chiama super, la superclasse deve avere un costruttore senza parametri
- Perché?

Esempio: Impossible Mission

- Abbiamo tanti “oggetti” (in senso lato)
  - Piattaforme
  - Computer
  - Oggetti in cui cercare indizi
- Alcuni sono “personaggi”
  - Il giocatore
  - I robot
  - Il “bombone”
- Che cosa hanno in comune tutti?
- E che cosa li distingue?







Modellare Impossible Mission in un quarto d'ora: la classe "astratta" Entità

- Abbiamo bisogno di una classe molto generale (quindi astratta) che rappresenti oggetti mobili e immobili nel gioco:

```
package it.uniroma1.impmiss;

abstract public class Entita
{
    protected int x;
    protected int y;

    public Entita(int x, int y)
    {
        this.x = x;
        this.y = y;
    }

    public int getX() { return x; }
    public int getY() { return y; }
}
```

Campi a **visibilità protetta**

- La visibilità protetta (protected) rende visibile il campo (o il metodo) a tutte le sottoclassi
  - Ma anche a tutte le classi del package (!)

Modellare Impossible Mission in un quarto d'ora:
gli oggetti

- Modelliamo gli oggetti immobili in astratto:

```
package it.uniroma1.impmiss;

abstract public class Oggetto extends Entita
{
    private TesseraPuzzle tessera;

    public Oggetto(int x, int y)
    {
        this(x, y, null);
    }

    public Oggetto(int x, int y, TesseraPuzzle tessera)
    {
        super(x, y);
        this.tessera = tessera;
    }

    public TesseraPuzzle search() { return tessera; }
}
```

Doppio costruttore (overloading)

Riuso del codice chiamando l'altro costruttore mediante la parola chiave this

Richiama il costruttore della superclasse (**OBBLIGATORIO** perché ha almeno un parametro) con la parola chiave super

Metodo aggiuntivo

Modellare Impossible Mission in un quarto d'ora:

gli oggetti

- che possono contenere una tessera del puzzle del gioco:

```
package it.uniroma1.impmiss;

public class TesseraPuzzle
{
    // da implementare
}
```

this e super

- La parola chiave this usata come nome di metodo **obbligatoriamente** nella **prima riga del costruttore** permette di richiamare un altro costruttore della stessa classe
- La parola chiave super usata come nome di metodo **obbligatoriamente** nella prima riga del costruttore permette di richiamare un costruttore della superclasse
- Ogni sottoclasse deve esplicitamente definire un costruttore se la superclasse **NON** fornisce un costruttore senza argomenti
  - Questo avviene se si definisce un costruttore **con almeno un argomento** ma non si definisce il costruttore senza argomenti
- **Nota bene:** anche le classi astratte possono avere costruttori! **Perché?**

Esempio di dinamica delle chiamate a costruttori

- Definiamo una piccola gerarchia di tre classi X, Y, Z:

```
public class X
{
    public X(int k)
    {
        System.out.println("X(int k)");
    }

    public X()
    {
        System.out.println("X()");
    }
}
```

```
public class Y extends X
{
    public Y(int k)
    {
        System.out.println("Y(int k)");
    }
}
```

```
public class Z extends Y
{
    public Z(int k)
    {
        super(k);
        System.out.println("Z(int k)");
    }

    public Z()
    {
        this(0);
        System.out.println("Z()");
    }

    public static void main(String[] args)
    {
        Z z = new Z();
    }
}
```

Disegnate il diagramma delle classi in UML

Esempio di dinamica delle chiamate a costruttori

- Definiamo una piccola gerarchia di tre classi X, Y, Z:

```java
public class X
{
    public X(int k)
    {
        System.out.println("X(int k)");
    }

    public X()
    {
        System.out.println("X()");
    }
}
```

```java
public class Y extends X
{
    public Y(int k)
    {
        System.out.println("Y(int k)");
    }
}
```

```java
public class Z extends Y
{
    public Z(int k)
    {
        super(k);
        System.out.println("Z(int k)");
    }

    public Z()
    {
        this(0);
        System.out.println("Z()");
    }

    public static void main(String[] args)
    {
        Z z = new Z();
    }
}
```

Nell'eseguire l'istruzione new Z(); abbiamo in output:

```
X()
Y(int k)
Z(int k)
Z()
```

Esempio di dinamica delle chiamate a costruttori

```
public class X
{
}

public class Y extends X
{
}
```

Costruttori di default in X e Y



```
public class X
{
	public X()
	{

	}
}

public class Y extends X
{
}
```

Costruttore esplicito con zero argomenti per X e di default per Y (quello di Y chiama automaticamente quello di X)



```
public class X
{
}

public class Y extends X
{
	public Y(int k)
	{

	}
}
```

Costruttore di default in X e con un argomento in Y (chiama in automatico il costruttore di default di X)



Esempio di dinamica delle chiamate a costruttori

```
public class X
{
        public X(int k)
        {

        }
}

public class Y extends X
{
}
```

Costruttore con un parametro per X e di default per Y



```
public class X
{
        public X()
        {

        }

        public X(int k)
        {

        }
}

public class Y extends X
{
}
```

Costruttori con zero e un parametro per X e di default per Y (quello di Y chiama automaticamente quello di X)



🎯Obiettivi

- ✅ ereditarietà
- ✅ classi/metodi astratti
- ✅ super (vs. this)
- ☐ overload/override
- ☐ is-a vs. has-a

Modellare Impossible Mission in un quarto d'ora:

un esempio di oggetto e il computer

- Modelliamo un possibile oggetto immobile:

```
package it.uniroma1.impmiss;

public class Libreria extends Oggetto
{
    public Libreria(int x, int y, TesseraPuzzle tessera)
    {
        super(x, y, tessera);
    }

    public Libreria(int x, int y)
    {
        super(x, y);
    }
}
```



```
package it.uniroma1.impmiss;

abstract public class Entita
{
    protected int x;
    protected int y;

    public Entita(int x, int y)
    {
        this.x = x;
        this.y = y;
    }

    public int getX() { return x; }
    public int getY() { return y; }
}
```

- e il computer:



```
package it.uniroma1.impmiss;

public class Computer extends Entita
{
    public Computer(int x, int y)
    {
        super(x, y);
    }

    public void login() { /* da implementare */ }
    public void logout() { /* da implementare */ }
}
```

Metodi aggiuntivi

```
package it.uniroma1.impmiss;

abstract public class Oggetto extends Entita
{
    private TesseraPuzzle tessera;

    public Oggetto(int x, int y)
    {
        this(x, y, null);
    }

    public Oggetto(int x, int y, TesseraPuzzle tessera)
    {
        super(x, y);
        this.tessera = tessera;
    }

    public TesseraPuzzle search() { return tessera; }
}
```

Modellare Impossible Mission in un quarto d'ora:

entità mobili:

- Modelliamo un generico personaggio:

```
package it.uniroma1.impmiss;

abstract public class Personaggio extends Entita
{
    public enum Direzione
    {
        DESTRA,
        SINISTRA,
        ALTO,
        BASSO;
    }

    private String nome;
    private int velocita;

    public Personaggio(int x, int y, String nome, int velocita)
    {
        super(x, y);
        this.nome = nome;
        this.velocita = velocita;
    }

    public String getNome() { return nome; }
    public int getVelocita() { return velocita; }

    public void muoviti(Direzione d)
    {
        switch(d)
        {
            case DESTRA: x += velocita; break;
            case SINISTRA: x -= velocita; break;
            // in futuro: emetti eccezione!
            default: System.out.println("Direzione non ammessa"); break;
        }
    }
}
```

Enumerazione delle direzioni

Campi aggiuntivi

Si possono fornire **implementazioni** nella classe astratta!

Metodi aggiuntivi

Modellare Impossible Mission in un quarto d'ora:

il giocatore

- Modelliamo il giocatore (ovvero la spia):

```
package it.uniroma1.impmiss;

public class Spia extends Personaggio
{
    public Spia(int x, int y, String nome, int velocita)
    {
        super(x, y, nome, velocita);
    }

    public void salta()
    {
        // ...
    }
}
```



nemico e robot

- Modelliamo un generico nemico:

Modellare Impossible Mission in un quarto d'ora:

nemico e robot

- Modelliamo un generico nemico:

```
package it.uniroma1.impmiss;

abstract public class Nemico extends Personaggio
{
    public Nemico(int x, int y, String nome, int velocita)
    {
        super(x, y, nome, velocita);
    }

    abstract public void attacca();
}
```

Metodo astratto!

- E i robot: 

```
package it.uniroma1.impmiss;

public class Robot extends Nemico
{
    public Robot(int x, int y, String nome, int velocita)
    {
        super(x, y, nome, velocita);
    }

    public void incenerisci() { /* fulmine elettrico */ }

    public void attacca() { /* da implementare */ }
}
```

Nella sottoclasse siamo **obbligati** a definire il metodo astratto

Modellare Impossible Mission in un quarto d'ora:

Il «bombone»

- Modelliamo un bombone:



```
package it.uniroma1.impmiss;

public class Bombone extends Nemico
{
    public Bombone(int x, int y, String nome, int velocita)
    {
        super(x, y, nome, velocita);
    }

    public void attacca()
    {
        // da implementare
    }

    public void muoviti(Direzione d)
    {
        switch(d)
        {
            case DESTRA: case SINISTRA: super.muoviti(d); break;
            case ALTO: y -= getVelocita(); break;
            case BASSO: y += getVelocita(); break;
        }
    }

    public void muoviti(Spia p)
    {
        muoviti(p.x > this.x ? Direzione.DESTRA : Direzione.SINISTRA);
        muoviti(p.y > this.y ? Direzione.ALTO : Direzione.BASSO);
    }
}
```

Overriding del metodo

Overloading del metodo

Esempi di riuso del codice

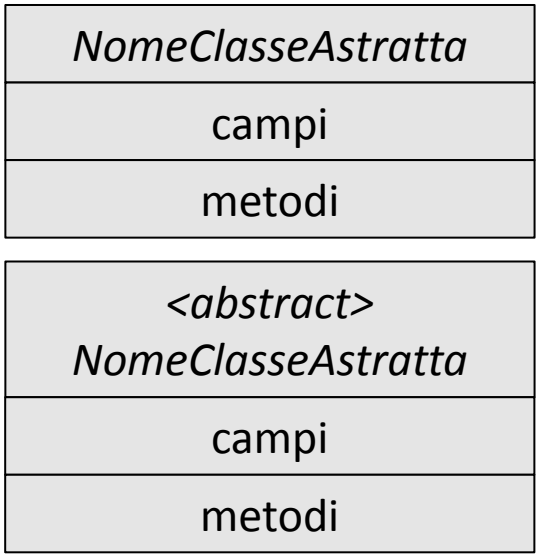
🎯Obiettivi

- ✅ ereditarietà
- ✅ classi/metodi astratti
- ✅ super (vs. this)
- ✅ overload/override
- ☐ is-a vs. has-a

Modellare Impossible Mission in un quarto d'ora:

Disegnate il diagramma delle classi risultante: Entita, Oggetto, TesseraPuzzle, Libreria, Computer, Personaggio, Spia, Nemico, Robot, Bombone

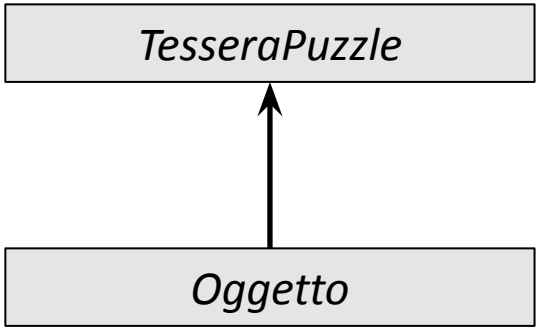
In UML una classe astratta ha il nome in corsivo e/o si usa il tag <abstract>



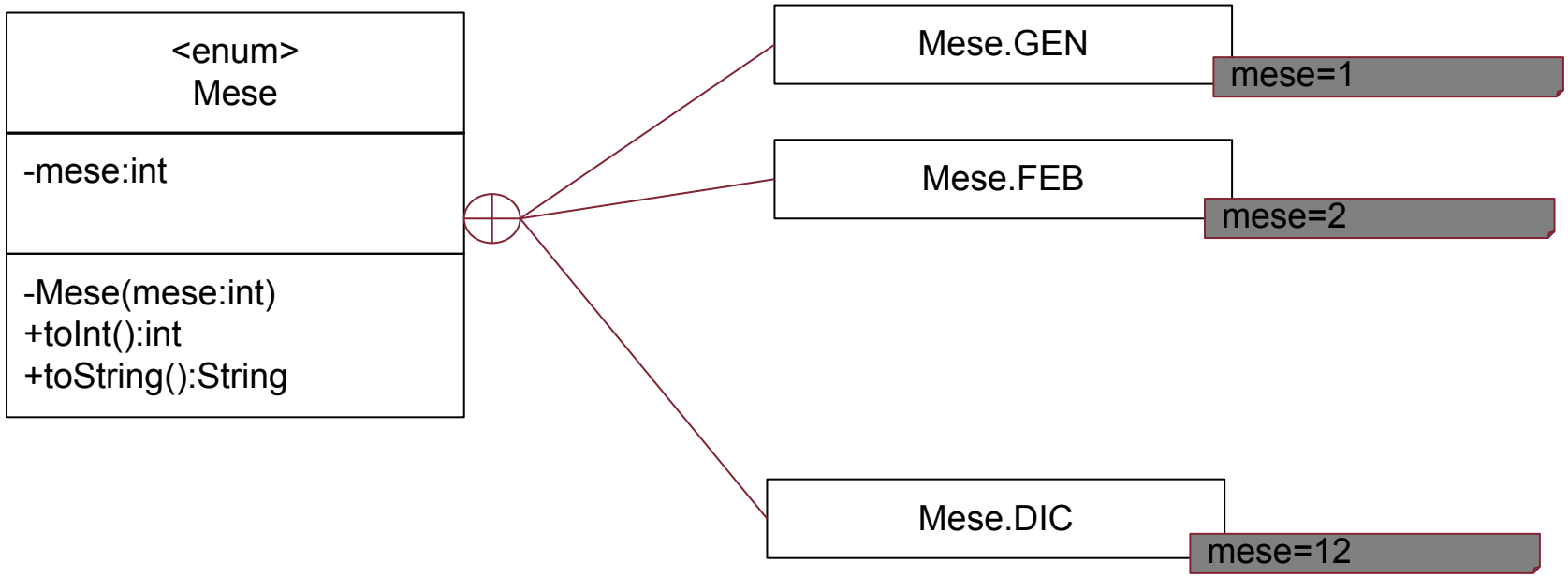
Modellare Impossible Mission in un quarto d'ora:

Disegnate il diagramma delle classi risultante: Entita, Oggetto, TesseraPuzzle, Libreria, Computer, Personaggio, Spia, Nemico, Robot, Bombone

In UML una dipendenza semplice si indica con una freccia



UML Parte 2


<enum>
Mese
-mese:int
-Mese(mese:int)
+toInt():int
+toString():String
Mese.GEN
mese=1
Mese.FEB
mese=2
Mese.DIC
mese=12

Differenza tra Overriding e Overloading

- L'overriding consiste nel ridefinire (reimplementare) un metodo con la stessa **intestazione** ("segnatura") presente in una superclasse
- L'overloading consiste nel creare un metodo con lo stesso nome, ma una **intestazione** diversa (diverso numero e/o tipo di parametri)

Mantenere il contratto

- Nell'overriding gli argomenti devono essere gli stessi
- I tipi di ritorno devono essere compatibili (lo stesso tipo o una sottoclasse)
- Non si può ridurre la visibilità (es. da public a private)

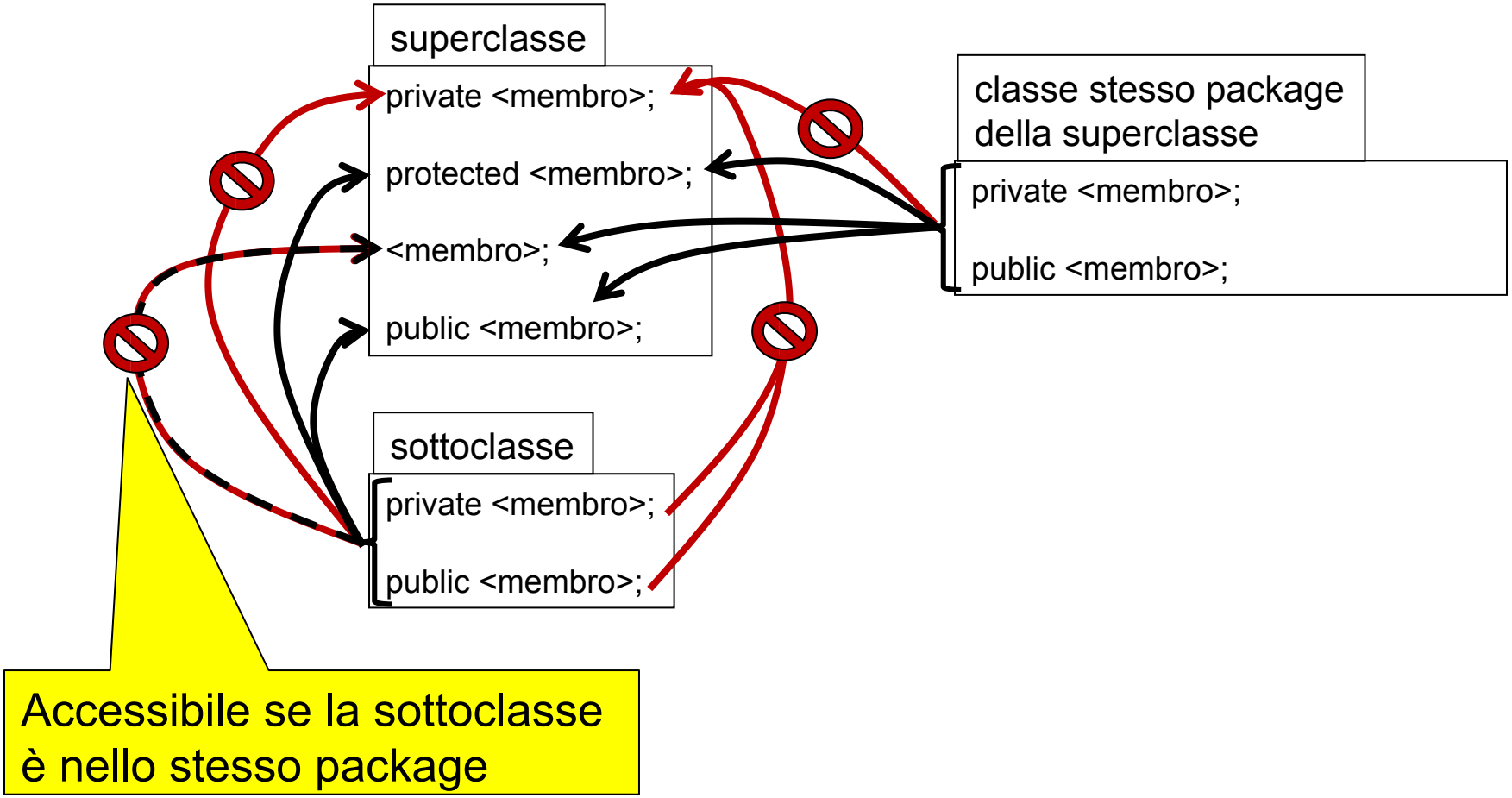
- Nell'overloading i tipi di ritorno possono essere diversi
  - **MA:** non si può cambiare **SOLO** il tipo di ritorno
- Si può variare la visibilità in qualsiasi direzione

Visibilità

Quattro possibilità per campi e metodi:

- Private: visibile solo all'interno della classe
- Public: visibile a tutti [all'interno di un modulo, vedi Java >=9]
- Default: visibile all'interno di tutte le classi del package
- Protected: visibile all'interno di tutte le classi del package e delle sottoclassi (indipendentemente dal package)

Visibilità


superclasse
private <membro>;
protected <membro>;
<membro>;
public <membro>;
classe stesso package
della superclasse
private <membro>;
public <membro>;
sottoclasse
private <membro>;
public <membro>;
Accessibile se la sottoclasse
è nello stesso package

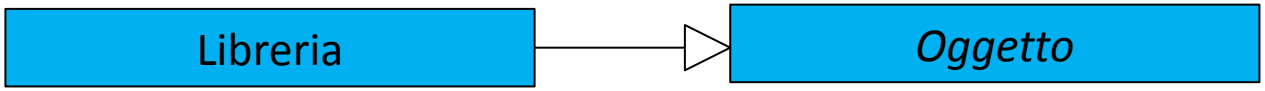
Is-a contro has-a



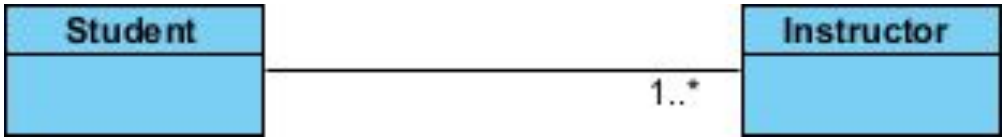
- E' **molto** importante distinguere tra relazioni di tipo è-un (is-a) e relazioni di tipo ha-un (has-a)
- Is-a rappresenta l'ereditarietà
  - Un oggetto di una sottoclasse può essere trattato come un oggetto della superclasse
  - **Domanda:** la sottoclasse è-un superclasse? (es. Paperino è un PersonaggioDisney? Sì! QuiQuoQua è un Paperino? No!)
- Has-a rappresenta la composizione
  - Un oggetto contiene come membri **riferimenti** ad altri oggetti
  - **Domanda:** un oggetto contiene altri oggetti? (es. Bagno contiene Vasca? Sì! PersonaggioDisney contiene Paperino? No!)

In UML

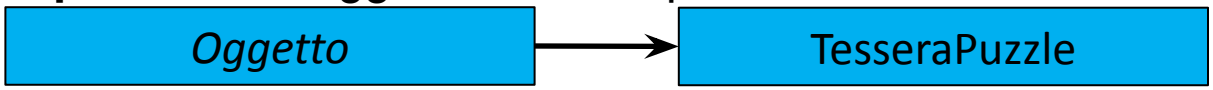
Is-A: implica una relazione gerarchica di ereditarietà



Associazione: implica una relazione generica che associa x oggetti di una classe a y oggetti di un'altra classe

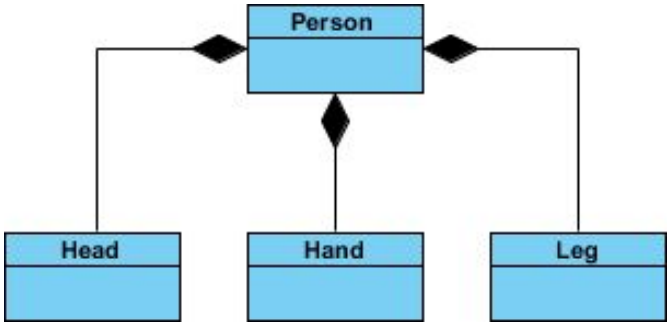


Dipendenza: Oggetto fa uso/dipende di/a TessaraPuzzle

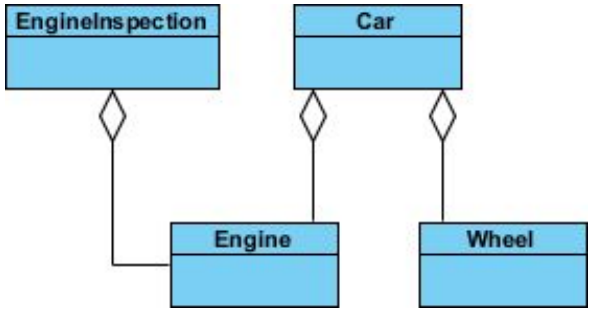


In UML

Composizione: implica una relazione dove un figlio non può esistere indipendentemente dal padre (ad es. Casa (padre) e Stanza (figlio))



Aggregazione: implica una relazione dove un figlio può esistere indipendentemente dal padre.



Esercizio: BarraDiEnergia & BarraDiEnergiaConPercentuale

- Creare una classe BarraDiEnergia costruita con un intero che ne determina la lunghezza massima. Inizialmente la barra è vuota. La barra è dotata di un metodo per l'incremento unitario del suo livello di riempimento e di un metodo toString che ne fornisca la rappresentazione sotto forma di stringa (es. se il livello è
  3 su 10, la stringa sarà "OOO=======".
- Creare quindi una seconda classe BarraDiEnergiaConPercentuale che fornisce una rappresentazione sotto forma di stringa come BarraDiEnergia ma stampando in coda alla stringa la percentuale del livello di riempimento. Per esempio, se il livello è 3 su 10, la stringa sarà "OOO======= 30%".

Esercizio: ListaDiInteri e ListaOrdinataDiInteri

Implementare una classe ListaDiInteri mediante un array (con i metodi specificati in fondo alle diapositive della terza parte: “controllo e array”)

- Implementare quindi una classe ListaOrdinataDiInteri per creare liste di interi ordinati in modo crescente. La classe ridefinisce i seguenti 3 metodi di aggiunta:
  - **aggiungiInCoda**(k): Aggiungi un intero in coda alla lista: l'aggiunta avviene solo se l'intero preserva l'ordine della lista.
  - **aggiungi**(k, j): Aggiungi un intero nella posizione specificata: come sopra, l'aggiunta avviene solo se l'intero preserva l'ordine degli interi della lista
  - **aggiungi**(k): Aggiungi un intero: l'intero viene inserito nella posizione appropriata, in modo da preservare l'ordine degli interi della lista
- L'array non deve essere ordinato con metodi di sorting, quali Arrays.sort

  (né vostri metodi di sorting *completo* dell'array)
- **Extra:** permettere di specificare un parametro da passare opzionalmente al costruttore di ListaOrdinataDiInteri per stabilire l'ordine della lista (crescente o decrescente; per default, crescente)

Esercizio: Animali

- Progettare (diagramma delle classi) ed implementare la classe Animale che rappresenti un generico animale
- La classe possiede i metodi emettiVerso() e getNumeroDiZampe()
- Possiede inoltre il metodo getTaglia() che restituisce un valore scelto tra: piccola, media e grande.
- Progettare (diagramma delle classi) ed implementare quindi le classi Mammifero, Felino, Gatto (taglia piccola), Tigre (grande), Cane, Chihuahua (piccola), Beagle (media), Terranova (grande), Uccello, Corvo (media), Passero (piccola), Millepiedi (piccola)
- Personalizzare in modo appropriato la taglia, il numero di zampe e il verso degli animali

Esercizio: Conto bancario

- Progettare la classe ContoBancario che rappresenti un conto con informazioni relative al denaro attualmente disponibile, il codice IBAN
- Modellare quindi una generica operazione bancaria Operazione che disponga di un metodo esegui()
- Modellare quindi i seguenti tipi di operazione:
  - PrelevaDenaro: preleva una specificata quantità di denaro da un dato conto
  - SvuotaConto: preleva tutto il denaro da un dato conto
  - VersaDenaro: versa del denaro sul conto specificato
  - SituazioneConto: stampa l'attuale saldo del conto
  - Bonifico: preleva del denaro da un conto e lo versa su un altro
- Specificare un metodo nella classe ContoBancario che restituisca l'elenco delle operazioni svolte in ordine temporale

Esercizio: Distributore automatico

- Progettare una classe Prodotto con un prezzo e tre tipi diversi di prodotto: BottigliaDAcqua, BarraDiCioccolato, GommeDaMasticare
- Progettare la classe DistributoreAutomatico che rappresenti un distributore automatico costruito con un intero N che determina il numero di prodotti nel distributore
- La classe prevede i seguenti metodi:
  - un metodo carica() che inserisce N prodotti di tipo e ordine casuale
  - un metodo inserisciImporto() che permette di inserire un importo nella macchinetta
  - un metodo getProdotto() che, dato in ingresso un numero di prodotto, restituisca il prodotto associato a quel numero e decrementi il saldo disponibile nel distributore
  - Un metodo getSaldo() che restituisca il saldo attuale del distributore
  - un metodo getResto() che restituisca il resto dovuto e azzeri il saldo

Esercizio: Espressioni matematiche

- Progettare una serie di classi che modellino le espressioni matematiche secondo la seguente definizione:
  - Una costante di tipo double è un'espressione
  - Una variabile con nome di tipo stringa e valore double è un'espressione
  - Se e_1 è un'espressione, allora $-e_1$ è un'espressione
  - Se e_1, e_2 sono espressioni, allora e_1 op e_2 è un'espressione dove op può essere l'operatore +, -, *, /, %

• Ogni tipo di espressione (costante, variabile, espressioni composte) deve essere modellata mediante una classe separata

• Ogni espressione dispone del metodo getValore() che restituisce il valore che quell'espressione possiede in quel momento

- Costruire quindi l'espressione –(5+(3/2)-2)*x e calcolarne il valore quando la variabile x vale 3 e quando la variabile x vale 6

- **Suggerimenti:** la variabile può modificare il proprio valore nel tempo; servirà veramente salvare un valore nella superclasse?

- **Alternative:** progettarlo usando l'ereditarietà (meglio) e mediante enum per le espressioni binarie

Esercizio: il Gioco dell'Oca

- Progettare il Gioco dell'Oca modellando:

– Il Giocatore, che mantiene l'informazione sulla posizione nel tabellone e i punti accumulati e implementa il metodo tiraDadi()

– Il Tabellone come sequenza di caselle costruita a partire da un intero N e da un elenco di giocatori; la classe dispone dell'operazione di posizionamento dei giocatori tenendo conto dell'effetto "gioco dell'oca" in cui, arrivati alla fine, si torna indietro

– Diversi tipi di caselle ciascuna con un diverso effetto a seguito del posizionamento del giocatore su quella casella:

– una CasellaVuota (nessun effetto sul giocatore)

– una CasellaSpostaGiocatore che sposta il giocatore di x caselle (avanti se x > 0 o indietro se x < 0)

– una CasellaPunti che ha l'effetto di far guadagnare o perdere un certo numero di punti al giocatore

– la classe GiocoDellOca che, dato un intero N e dati i giocatori, che inizializza un tabellone di lunghezza N e implementa il metodo giocaUnTurno() che fa effettuare una mossa a ognuno dei giocatori

Esercizio: Tetris

- Progettare il gioco del Tetris modellando la classe Pezzo con le seguenti operazioni:
  - Left : sposta a sinistra il pezzo – Right: sposta a destra il pezzo
  - Rotate: ruota il pezzo in senso orario
  - Down: manda giù il pezzo
- Progettare anche la classe di ciascun pezzo (a forma di L, a forma di T, a serpente, a forma di I e cubo)
- Progettare infine la classe Tetris che somministra i pezzi, permette al giocatore di muoverli, gestisce lo spazio libero e calcola i punteggi del giocatore

🎯Obiettivi

- ✅ ereditarietà
- ✅ classi/metodi astratti
- ✅ super (vs. this)
- ✅ overload/override
- ✅ is-a vs. has-a

Credits

Le slide di questo corso sono basate su quelle utilizzate dal Prof. Faralli, le quali sono a loro volta una rielaborazione delle slide del Prof. Navigli, e frutto della revisione degli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica Mario Marra e Paolo Straniero.